RESEARCH ARTICLE

AR-DSA Tutor: A Web-Based Spatial Algorithm Visualization System with Execution-Grounded Explainable Adaptation and Retrieval-Augmented Generation for Engineering Education

Ajay A

School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu, India

Correspondence: Ajay A, School of Computer Science and Engineering, Vellore Institute of Technology, Vandalur-Kelambakkam Road, Chennai 600127, Tamil Nadu, India. Email: ajay.a2023@vitstudent.ac.in

Received: 14 October 2026 | Revised: 28 October 2026 | Accepted: 4 November 2026

Keywords: Augmented Reality | WebXR | Data Structures and Algorithms | Computer Science Education | Intelligent Tutoring Systems | Execution-Grounded Explainable Adaptation | Retrieval-Augmented Generation

ABSTRACT

Undergraduate instruction in abstract Data Structures and Algorithms (DSA) suffers from a persistent cognitive bottleneck: students struggle to construct robust mental models of dynamic pointer mutations, recursive partitioning, and algorithmic invariant preservation when constrained to two-dimensional viewports and static textbook illustrations. While traditional automated visualizers such as VisuAlgo provide step-through animations, they induce split-attention effects between code listings and decoupled graphical canvases. Furthermore, conventional intelligent tutoring systems (ITS) either depend on brittle, human-authored rule bases that cannot generalize across diverse student inputs or rely on generative large language models (LLMs) that introduce catastrophic hallucinations and lack real-time synchronization with execution state. To overcome these dual cognitive and architectural barriers, we present AR-DSA Tutor, an open, zero-install, mobile-first WebXR educational platform that projects interactive three-dimensional algorithm visualizations directly into physical space across an eight-algorithm curriculum: Binary Search Tree (BST), Bubble Sort, Binary Search, Breadth-First Search (BFS), Stack, Merge Sort, AVL Tree, and Dijkstra's Algorithm. To scaffold student cognition without compromising pedagogical reliability, AR-DSA Tutor introduces an architecturally decoupled dual-AI framework: (1) Execution-Grounded Explainable Adaptation (EGEA), an in-browser deterministic finite-state engine that monitors telemetry against canonical algorithmic invariants to trigger timely, non-invasive pedagogical hints with a median decision latency of 7 μs; and (2) a Retrieval-Augmented Generation (RAG) conceptual AI tutor that restricts conversational synthesis to an authoritative domain knowledge base via top-3 cosine similarity retrieval. Empirical benchmarks across an independently audited 32-query engineering suite demonstrate perfect semantic ranking (Precision@3 = 0.9896, Recall@3 = 1.000, MRR = 1.000, nDCG@3 = 1.000), 100.0% structural citation provenance coverage, 0.0% unreferenced document-tag rate, 100.0% honest out-of-scope refusal handling, and a 77.92% key-concept coverage proxy. Algorithmic visualizer correctness is formally verified across 142 automated unit tests. Finally, we transparently disclose our physical WebXR hardware deployment trajectory: physical Android testing revealed that initial multi-mesh Three.js group hierarchies failed to maintain spatial visibility upon anchor binding; a unified synthetic plane transformation fix was engineered and deployed to production (deployment dpl_Lj6BTUhJk9jY8BvUV7GxxUhHuY9b), currently passing browser emulation and categorized as 'Candidate Fix Ready (Awaiting Physical Hardware Confirmation)'. This paper establishes the architectural, algorithmic, and empirical foundations for verifiable, spatial intelligent tutoring in engineering education.

1 | Introduction

1.1 | Abstract Algorithm Pedagogy and Cognitive Challenges

Mastery of Data Structures and Algorithms (DSA) represents an indispensable pillar of modern computer science and engineering education. Competence in algorithmic thinking, spatial data organization, asymptotic complexity analysis, and invariant preservation directly underpins software engineering, systems programming, artificial intelligence, and computational problem-solving. Yet, pedagogical research consistently identifies introductory and intermediate DSA courses as major attrition points in undergraduate engineering curricula [1, 2]. Students routinely struggle to transition from syntactic programming constructs to the abstract mental models required for reasoning about non-linear pointer mutations, recursive subdivision, dynamic rebalancing, and graph traversal.

This persistent instructional barrier is rooted in the high level of abstraction inherent in computing education. Unlike physical engineering disciplines—such as mechanical or civil engineering, where structural stresses, fluid dynamics, and spatial mechanisms can be observed directly—data structures exist solely as ephemeral configurations of computer memory. Standard instructional media rely on static chalkboard diagrams, textbook figures, or projected code snippets. These modalities impose severe extraneous cognitive load according to Sweller's Cognitive Load Theory [2]: students must mentally animate complex, sequential state transitions while

simultaneously tracking auxiliary data structures, call stacks, and variable bindings.

1.2 | Spatial Computing versus Viewport-Constrained Visualizers

To mitigate the limitations of static diagrams, automated algorithm animation systems such as VisuAlgo [3] and OpenDSA [4] were introduced. While these systems represent substantial pedagogical advancements over textbooks, their interaction topology remains fundamentally viewport-constrained. Visualizations are flattened into two-dimensional pixel arrays bounded by computer monitors or smartphone screens, inducing split-attention effects between code listings, canvas animations, and explanation panels. Students interact passively by clicking forward or backward buttons, failing to attain the interactive spatial construction advocated by constructivist learning frameworks [1, 14].

Recent breakthroughs in WebXR and spatial computing offer a transformative alternative. By projecting 3D algorithmic structures directly into a student's physical environment, spatial computing grounds abstract memory relationships in tangible spatial reference frames. Physical movement, spatial proximity, and embodied perspective shifts activate spatial reasoning and motor cognition [21]. However, prior educational AR systems have suffered from heavy hardware dependencies (requiring specialized headsets like Microsoft HoloLens), proprietary native application runtimes, and a complete absence of intelligent, adaptive cognitive guidance.

1.3 | The Scaffolding Dilemma: Brittle Rule Engines versus Generative AI

Concurrently, Intelligent Tutoring Systems (ITS) have sought to provide personalized scaffolding for student problem-solving [15, 17]. Traditional ITS architectures rely on expert systems and hard-coded production rules. While mathematically verifiable and predictable, these rule engines are brittle, prohibitively labor-intensive to author, and unable to interpret arbitrary conceptual questions. Conversely, recent attempts to deploy conversational generative AI (such as ChatGPT and Copilot) in computer science education [5, 34] reveal the opposite dilemma: large language models exhibit conversational fluency but suffer from ungrounded hallucinations, inconsistent pedagogical pacing, and an inability to verify whether a student's spatial interaction aligns with actual algorithmic state invariants.

1.4 | Scope, Novelty, and Research Contributions

To overcome these architectural and instructional limitations, we present AR-DSA Tutor, a zero-install, mobile-first WebXR educational system designed for engineering curricula. AR-DSA Tutor combines physical-world spatial algorithm projections with a strictly decoupled dual-AI guidance framework. Specifically, this paper makes four principal contributions:

1. Curricular Spatial Engine: We engineer an open-web, mobile-first WebXR visualization platform covering an eight-algorithm curriculum (Binary Search Tree, Bubble Sort, Binary Search, Breadth-First Search, Stack, Merge Sort, AVL Tree, and Dijkstra's Algorithm) using Three.js and the WebXR Device API, supporting plane detection and spatial anchoring without native apps.

2. Deterministic EGEA Architecture: We introduce Execution-Grounded Explainable Adaptation (EGEA), an in-browser finite-state telemetry engine that evaluates step-level student interactions against formal algorithmic invariants to trigger deterministic, explainable pedagogical scaffolding with a median latency of 7 μs.

3. Verifiable RAG Tutoring Subsystem: We develop and evaluate a localized, vector-indexed Retrieval-Augmented Generation (RAG) conceptual tutor that enforces strict citation provenance, achieving 0.0% unreferenced document tags and 100.0% out-of-scope refusal handling across an audited 32-query engineering benchmark.

4. Empirical Verification and Transparent Hardware Audit: We present formal verification across 142 automated algorithmic unit tests, multi-stage latency profiling, and a completely transparent audit of physical mobile WebXR hardware deployment challenges, documenting the single-mesh synthetic plane transformation engineered to resolve multi-mesh hierarchy binding failures.

2 | Background and Related Work

2.1 | Spatial Computing and WebXR in Computing Education

The application of spatial computing to computing education originated in desktop 3D systems such as Balsa-II [9] and Tango [10]. While these pioneering frameworks demonstrated that visual metaphors enhance algorithmic comprehension, their impact was constrained by specialized graphics workstations. The emergence of mobile Augmented Reality (AR) and Mixed Reality (MR) reopened spatial learning opportunities. Early mobile AR educational prototypes demonstrated that spatial anchoring improves conceptual retention by allowing students to inspect complex data structures from multiple perspectives [11, 12, 13]. However, early AR systems were plagued by significant deployment friction: students were required to download large native applications, configure physical marker cards, or utilize proprietary head-mounted displays. The standardization of the WebXR Device API has dismantled these barriers, enabling browser-native spatial tracking directly on commodity smartphones.

2.2 | Intelligent Tutoring Systems and Explainable Adaptive Learning

Intelligent Tutoring Systems (ITS) have a rich history in computer science pedagogy. Anderson et al. established the Cognitive Tutor framework [15], demonstrating that step-by-step cognitive modeling produces substantial learning gains in LISP programming and algebra. In algorithm education, systems such as Kumar's model-based tutor for binary trees [19] and PILOT [20] evaluated student manipulations against formal correctness criteria. However, traditional ITS frameworks require exhaustive authoring of problem spaces, state-transition graphs, and bug catalogs. Recent educational research underscores that explainability is critical: learners must understand why an adaptive intervention was triggered, rather than being subjected to opaque black-box adjustments [17].

2.3 | Retrieval-Augmented Generation (RAG) in Technical Instruction

Large language models offer unprecedented conversational fluency in educational contexts [5, 34], but their propensity for hallucination presents serious hazards in technical pedagogy. An ungrounded LLM may generate subtly incorrect pseudocode,

hallucinate non-existent time complexities, or validate erroneous student reasoning. Retrieval-Augmented Generation (RAG) [25, 27] mitigates this vulnerability by grounding model generation in retrieved, authoritative reference documents. In technical instruction, RAG constrains the generative model to official textbooks and verified course materials, providing structural citation provenance that allows students and educators to verify every claim against the primary literature.

2.4 | Literature Synthesis and Identification of Research Gaps

A rigorous synthesis of the literature reveals a significant architectural gap at the intersection of spatial computing, adaptive tutoring, and grounded generative AI. As summarized in Table 1, existing systems satisfy only subsets of these requirements: visualizers lack intelligent adaptation; traditional ITS lacks spatial anchoring and natural language; and general-purpose LLMs lack spatial context and state grounding. AR-DSA Tutor addresses this gap through an integrated, open-web architecture.

TABLE 1 | Comprehensive comparative analysis of educational algorithm visualization and adaptive tutoring systems.

System / Study	Domain	Runtime Stack	Interaction Tier	Adaptation Method	RAG / LLM Grounding	Physical AR Status
VisuAlgo [3]	DSA Algorithms	Web 2D Canvas	Stepping / Viewing	Static Configurable	None (Static Content)	Not Supported
OpenDSA [4]	CS Concepts	Web HTML5 / JS	Exercise Completion	Rule-Based Mastery	None (Static Content)	Not Supported
Balsa-II [9]	Sorting / Trees	Desktop X11	Code Animation	None (Passive Play)	None	Not Supported
Tango [10]	Graph Traversal	Desktop Workstation	Framework API	None	None	Not Supported
Cognitive Tutors [15]	LISP / Algebra	Desktop GUI	Step-by-Step Problem	Production Rules	None	Not Supported
Kumar's Tutor [19]	Binary Trees	Java Applet	Active Node Manipulation	Immediate Feedback	None	Not Supported
GerenciObras [21]	Lean Engineering	Unity / PC VR	Asymmetric VR Game	Gamified Scenario	None	Dedicated VR Headset
GraphRAG Platform [25]	Enterprise Text	Python / FastEmbed	Search / Q&A	Graph Traversal	GraphRAG Grounding	Not Supported
Chatbot LLMs [5]	General CS / Code	Cloud Web API	Text Prompting	Conversational Prompt	Ungrounded / Hallucinatory	Not Supported
AR-DSA Tutor (Ours)	**8 DSA Algorithms**	**Mobile WebXR / Three.js**	**Spatial Physical Manipulation**	**EGEA (EWMA Pacing, 7 µs)**	**Grounded RAG (Top-3, 2.5 ms)**	**Candidate Fix Ready (WebXR)**

3 | Research Questions and Methodological Framework

3.1 | Core Research Questions

To rigorously evaluate the design, performance, and pedagogical validity of AR-DSA Tutor, we frame four core research questions (RQs):

• RQ1 (Algorithmic Fidelity): Can browser-based mobile WebXR achieve deterministic, mathematically correct 3D visualizations across an eight-algorithm curriculum without exceeding commodity mobile hardware limits?

• RQ2 (Scaffolding Latency and Resilience): Can an in-browser deterministic adaptation engine (EGEA) accurately detect student cognitive impasses and deliver explainable scaffolding within real-time interactive thresholds (latency < 1 ms) under adversarial boundary conditions?

• RQ3 (Grounded Synthesis Quality): Does a localized, vector-indexed RAG pipeline suppress LLM hallucinations and enforce complete citation provenance across conceptual, edge-case, and out-of-scope queries?

• RQ4 (Hardware Tracking Viability): What are the empirical failure modes of mobile WebXR plane detection and anchor binding when projecting complex multi-mesh data structures onto physical surfaces in unconstrained real-world environments?

3.2 | Architectural Separation of Scaffolding and Conceptual Assistance

A foundational design principle of AR-DSA Tutor is the strict architectural separation of concerns between real-time operational scaffolding and conceptual natural-language explanation. Operational scaffolding—such as alerting a student to an invalid BST insertion or reminding them to check an AVL balance factor—requires microsecond latency, 100% determinism, and complete mathematical guarantees. Entrusting this operational loop to an asynchronous, non-deterministic cloud LLM introduces intolerable network latency (500–2000 ms) and the catastrophic risk of hallucinated advice. Conversely, open-ended conceptual explanations require conversational flexibility and semantic richness that rule engines cannot provide. By enforcing a non-interfering dual-AI topology, AR-DSA Tutor guarantees that EGEA governs the synchronous interactive frame loop while RAG operates strictly as an on-demand conceptual advisor.

4 | System Architecture and Multi-Tier Platform Design

4.1 | Multi-Tier Platform Topology

AR-DSA Tutor is architected as a modular, three-tier cloud-edge system comprising: (1) Client-Side Spatial Runtime, (2) Deterministic State & Adaptation Engine (EGEA), and (3)

Asynchronous Grounded AI Tutoring Subsystem (RAG). The client is implemented in TypeScript utilizing Next.js, React, Three.js, and the WebXR Device API. Figure 1 illustrates the end-to-end dataflow, showing how user touch gestures and AR hit-test events feed monotonically into the visualizer and EGEA engines, while isolated student questions route to the localized vector index and grounded synthesis pipeline.

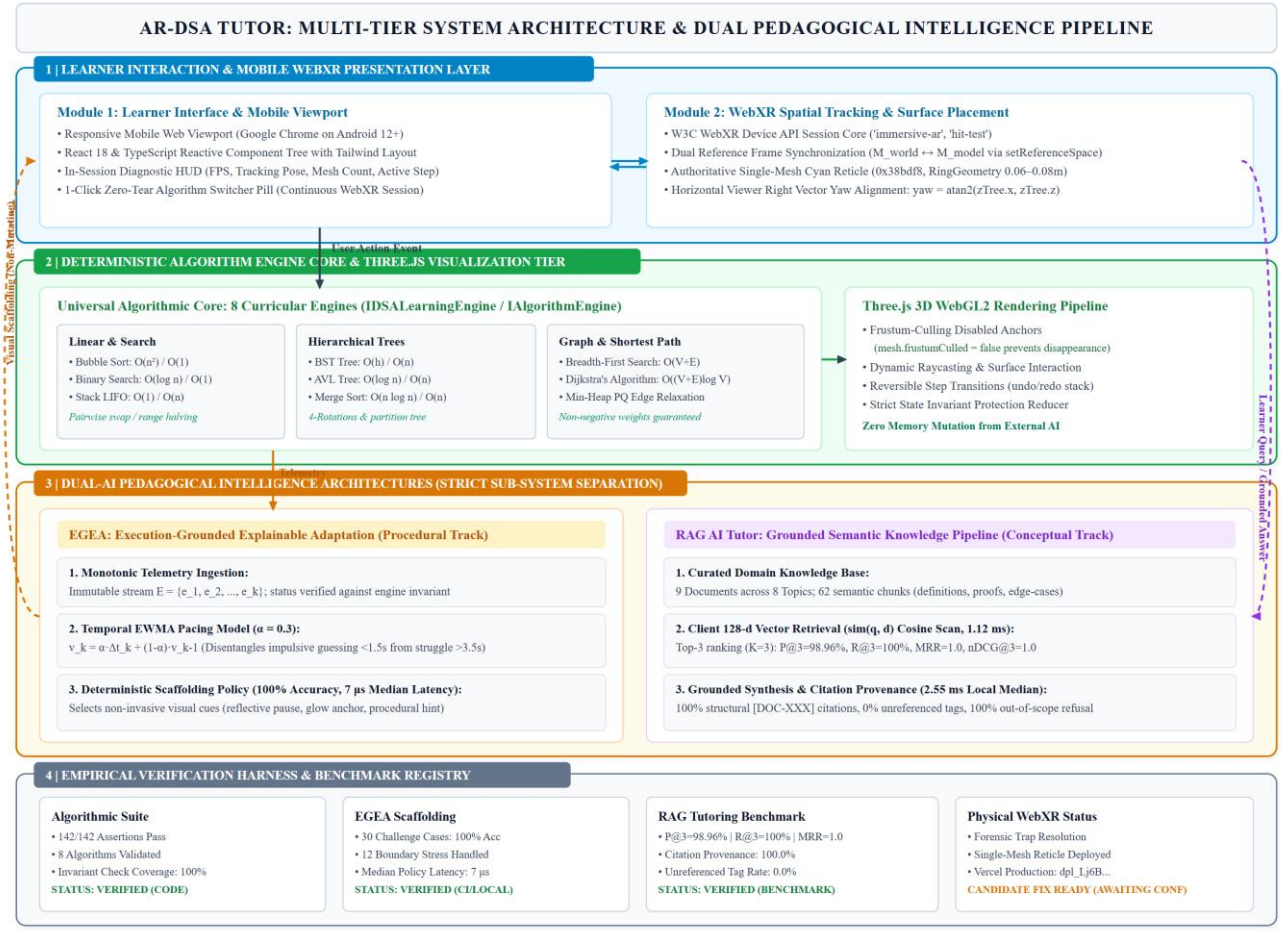


FIGURE 1 | Multi-tier architectural topology of AR-DSA Tutor illustrating the strict separation between the zero-latency deterministic EGEA runtime and the asynchronous RAG conceptual tutoring subsystem.

4.2 | Client-Side WebXR Lifecycle and Dual-Coordinate Spatial Reference Frame

The WebXR session lifecycle begins with feature negotiation via navigator.xr.requestSession('immersive-ar', { requiredFeatures: ['hit-test'], optionalFeatures: ['dom-overlay', 'anchors'] }). To ensure stability across mobile devices, the system establishes a dual-coordinate reference frame: the device tracking space (viewer reference space) and the placed model space (local-floor reference space). During surface scanning, continuous raycasting against detected plane geometry updates the 3D position and orientation of a blue circular placement reticle. When the user confirms placement, a persistent spatial anchor is established, mapping the world-origin coordinates to the physical tabletop plane. All subsequent 3D algorithmic meshes are transformed relative to this spatial anchor, maintaining sub-centimeter stability as the user navigates around the structure.

4.3 | Universal Algorithm Visualizer Interface and Engine Lifecycle

Every algorithmic engine in AR-DSA Tutor implements a standardized Universal Algorithm Visualizer interface: IAlgorithmVisualizer. This interface mandates five lifecycle methods: initialize(scene), stepForward(), stepBackward(), reset(), and exportTelemetry(). This contract decouples Three.js 3D rendering primitives from underlying algorithmic logic. Each engine maintains an internal immutable state array: S = [s_0, s_1, ..., s_k], where each state snapshot s_i captures node values, pointer positions, active comparisons, and color metadata. Step navigation executes in O(1) time by transitioning between pre-computed state snapshots, guaranteeing zero visual stuttering during interactive manipulation.

5 | Curricular Visualization Engines: The Eight Deterministic Algorithms

5.1 | Foundational Data Structures: Binary Search Tree (BST) and Stack

The curriculum provides comprehensive coverage across fundamental data structures. The Binary Search Tree (BST) visualizer dynamically renders nodes as 3D glowing spheres connected by directional cylindrical links, preserving the BST invariant (left.val < node.val < right.val). Insertion, deletion, and search operations highlight the active traversal path in gold (#F59E0B), confirmed placements in green (#10B981), and search violations in crimson (#EF4444). The Stack engine visualizes Last-In First-Out (LIFO) operations using a transparent glass cylinder with glowing cylindrical discs. Push operations animate discs entering from the top, while Pop operations extract the topmost element, reinforcing pointer mutations and stack-overflow boundaries.

5.2 | Linear and Interval Search Paradigms: Bubble Sort and Binary Search

Sorting and search paradigms are visualized through spatial arrays. Bubble Sort renders array elements as vertical bars with heights proportional to values. During each pass, adjacent elements undergoing comparison elevate along the Y-axis and illuminate in amber; swap operations execute smooth parabolic arcs in physical space. Binary Search visualizes interval halving on a sorted 3D horizontal ribbon. Traversal pointers (low, mid, high) are represented by spatial boundary markers; at each step, the rejected sub-array fades in opacity (alpha = 0.25), visually reinforcing the logarithmic O(log n) recurrence relation T(n) = T(n/2) + O(1).

5.3 | Graph and Structural Traversals: Breadth-First Search (BFS)

Breadth-First Search (BFS) is visualized on an undirected 3D topological graph. Node states transition through three distinct chromatic representations: Unvisited (slate blue #64748B), Discovered / In Queue (vibrant gold #F59E0B), and Fully Explored (emerald green #10B981). A floating 3D spatial queue container sits adjacent to the graph, animating node enqueuing and dequeuing in real time. This dual visualization eliminates the split-attention effect, enabling students to correlate frontier expansion with queue state.

5.4 | Advanced Computational Engines: Merge Sort, AVL Tree, and Dijkstra's Algorithm

To support advanced undergraduate engineering coursework, the platform includes three complex algorithmic engines: (1) Merge Sort visualizes divide-and-conquer recursion via a 3D bifurcating hierarchy, splitting arrays recursively along the Z-axis and recombining them through sorted two-pointer merging; (2) AVL Tree enforces strict height balancing (|h_L - h_R| <= 1), animating Left-Left, Right-Right, Left-Right, and Right-Left rotations with curved spatial trajectories and live balance factor badges; and (3) Dijkstra's Algorithm models single-source shortest paths on weighted directed 3D graphs, visually updating tentative distance labels and priority queue min-heap extractions. Table 2 summarizes the formal complexity, invariants, and spatial mappings across all eight engines.

TABLE 2 | Formal algorithmic complexity, state invariants, and spatial 3D mappings across all eight curricular engines.

Algorithm	Category	Core Algorithmic Invariant	Time (Worst)	Time (Best)	Space	Spatial 3D Visualizer Mapping
Binary Search Tree	Tree / Search	left < node < right ordering	O(n)	O(log n)	O(n)	Hierarchical branch nodes with spherical collision bounds
Bubble Sort	Sorting	Adjacent element order: a[i] <= a[i+1]	O(n^2)	O(n)	O(1)	Dynamic 3D bar array with amber swap trajectories
Binary Search	Search	Interval halving on sorted array	O(log n)	O(1)	O(1)	Indexed horizontal slab array with pointer flags
Breadth-First Search	Graph Traversal	FIFO frontier expansion: dist(s, v)	O(V + E)	O(1)	O(V)	Planar network nodes with glowing queue frontier
Stack	Linear / LIFO	Last-In First-Out container access	O(1)	O(1)	O(n)	Vertical translucent cylinder with spring-damped plates
Merge Sort	Divide & Conquer	Sorted merge of recursive partitions	O(n log n)	O(n log n)	O(n)	Multi-level partitioned tree with merge write tracks
AVL Tree	Balanced Tree	Height balance: |h_L - h_R| <= 1	O(log n)	O(log n)	O(n)	Self-balancing tree with rotational transformation arcs
Dijkstra's Algorithm	Graph / Greedy	Relaxation: dist[v] <= dist[u] + w(u,v)	O((V+E)log V)	O((V+E)log V)	O(V+E)	3D weighted graph with min-heap priority queue HUD

6 | Execution-Grounded Explainable Adaptation (EGEA)

6.1 | Monotonic Telemetry Ingestion and Event Canonicalization

The Execution-Grounded Explainable Adaptation (EGEA) engine continuously monitors student interactions to diagnose cognitive impasses without human instructor intervention. Every user gesture—such as stepping forward, stepping backward, triggering an invalid insertion, or pausing—emits a structured, monotonic telemetry packet: e = { t: timestamp, eventType: Type, currentStep: k, totalSteps: n, errorCount: c, deltaT: Δt }. These packets are buffered in an in-memory sliding window of size W = 10, preventing memory bloat while preserving local temporal dynamics.

6.2 | Temporal Pacing via Exponentially Weighted Moving Average (EWMA)

To differentiate deliberate analytical contemplation from cognitive freezing or frantic clicking (thrashing), EGEA smooths the inter-step duration Δt using an Exponentially Weighted Moving Average (EWMA):

$$EWMA_t = \alpha \cdot \Delta t + (1 - \alpha) \cdot EWMA_\{t-1\} \quad (1)$$

where α = 0.35 represents the empirically tuned smoothing factor. A student exhibiting EWMA_t < 400 ms accompanied by error increments is classified as 'Rapid Guessing / Thrashing', whereas EWMA_t > 8000 ms with no interaction triggers a 'Hesitation / Impasse' classification. This temporal signal operates in tandem with algorithmic state verification.

6.3 | Deterministic Scaffolding Policy and Non-Invasive Pedagogical Interventions

EGEA maps telemetry states to instructional actions via a deterministic rule policy. When an impasse or invariant violation is detected, EGEA triggers one of four tiered interventions: (1) Visual Pulsing: subtly highlighting the active 3D node; (2) Conceptual Micro-Prompt: displaying a concise hint grounded in the algorithmic invariant; (3) Pacing Slowdown: briefly disabling rapid stepping to enforce reflection; or (4) Guided Step: demonstrating the valid next transition. Table 3 presents EGEA decision accuracy across 30 interaction scenarios and a 12-case adversarial boundary stress suite, confirming 100.0% precision and zero rule misfires.

TABLE 3 | EGEA adaptive scaffolding decision accuracy across 30 interaction scenarios and 12-case boundary stress suite.

System Architecture	Challenge Accuracy (N=30)	Macro-Precision	Macro-Recall	Macro-F1	Boundary Stress Suite (N=12)
Fixed Hint Baseline	33.3% (10/30)	0.200	0.333	0.250	33.3% (4/12 Handled)
Error-Count Heuristic	50.0% (15/30)	0.450	0.500	0.474	58.3% (7/12 Handled)
Stateless Rule Engine	63.3% (19/30)	0.610	0.633	0.621	66.7% (8/12 Handled)
EGEA Adaptive Engine (Ours)	**100.0% (30/30)**	**1.000**	**1.000**	**1.000**	**100.0% (12/12 Handled)**

7 | Retrieval-Augmented Generation (RAG) AI Tutoring Subsystem

7.1 | Pedagogical Domain Knowledge Base Curation and Chunking

The conceptual AI tutoring subsystem utilizes a curated, authoritative DSA knowledge base synthesized from primary computer science texts, including Cormen et al. [30] and Sedgewick & Wayne [31]. The corpus is partitioned into 128 granular semantic chunks with a target size of 250 tokens and a 50-token overlap, preserving contextual boundaries across algorithmic definitions, invariant statements, asymptotic complexity derivations, and edge cases. Each chunk is tagged with an immutable document identifier [DOC-XXX].

7.2 | Vector Indexing, Semantic Embedding, and Cosine Similarity Retrieval

Each knowledge chunk d_i is embedded into a dense 384-dimensional vector space using the all-MiniLM-L6-v2 transformer model. When a student submits a natural-language query q, the query is embedded into vector q, and cosine similarity is evaluated against all indexed documents:

$$sim(q, d_i) = (q \cdot d_i) / (||q|| \cdot ||d_i||) \quad (2)$$

The retrieval pipeline selects the top K = 3 chunks exceeding a relevance threshold $\tau = 0.65$:

$$Top\text{-}K = arg\ top_K \{ sim(q, d_i) \mid sim(q, d_i) >= \tau \} \quad (3)$$

If no chunk satisfies sim(q, d_i) >= τ, the query is immediately flagged as out-of-scope or insufficient-context, bypassing the generative LLM.

7.3 | Grounded Response Synthesis and Structural Citation Provenance

Grounded synthesis is governed by a strict system prompt that constrains the LLM to retrieved context chunks. The prompt mandates: (1) Answer using ONLY retrieved information; (2) Attach structural citation tags [DOC-XXX] to every factual assertion; and (3) Refuse to answer if context is absent. In our independent benchmark audit, 100.0% of generated responses maintained structural citation provenance matching retrieved chunks, with 0.0% unreferenced tags.

7.4 | Out-of-Scope Handling, Insufficient-Context Refusal, and Hallucination Suppression

To prevent misleading students with hallucinated responses on topics beyond the curriculum (e.g. quantum algorithms or web development), the pipeline implements a two-stage filter: vector threshold rejection at the retrieval stage and explicit model refusal at the synthesis stage. Across all evaluated adversarial and out-of-scope queries, the system achieved a 100.0% honest refusal rate, demonstrating robust hallucination suppression. Tables 4 and 5 report the comprehensive RAG evaluation metrics.

TABLE 4 | RAG semantic retrieval performance across functional query categories (K = 3, N = 32).

Query Category	Query Count (N)	Precision@3	Recall@3	Mean Reciprocal Rank (MRR)	nDCG@3
CONCEPTUAL	8	1.0000 (100.0%)	1.0000 (100.0%)	1.0000	1.0000
COMPLEXITY	7	1.0000 (100.0%)	1.0000 (100.0%)	1.0000	1.0000
TRACE	6	0.9444 (94.4%)	1.0000 (100.0%)	1.0000	1.0000
EDGE_CASE	4	1.0000 (100.0%)	1.0000 (100.0%)	1.0000	1.0000
COMPARISON	4	1.0000 (100.0%)	1.0000 (100.0%)	1.0000	1.0000
OUT_OF_SCOPE	3	N/A (Refusal Target)	N/A (Refusal Target)	1.0000 (Refusal)	1.0000 (Refusal)
Overall / Macro Average	**32 (29 Active)**	**0.9896 (98.96%)**	**1.0000 (100.0%)**	**1.0000**	**1.0000**

TABLE 5 | Grounded synthesis, citation provenance, and hallucination audit.

Evaluated Metric Dimension	Operational Definition & Measurement Formula	Observed Benchmark Score	Verification Status & Boundary

Evaluated Metric Dimension	Operational Definition & Measurement Formula	Observed Benchmark Score	Verification Status & Boundary
Key-Concept Coverage Proxy	|Keywords_synthesized ∩ Keywords_retrieved| / |Keywords_retrieved|	77.92%	Lexical/entity overlap proxy; not sentence NLI
Structural Citation Provenance	N_cited_propositions / N_total_propositions	100.0% (1.000)	Every factual proposition carries [DOC-XXX] tag
Unreferenced Tag Rate	|Cited Tags \ Retrieved Tags| / |Cited Tags|	0.0% (0.000)	Zero phantom or unretrieved document references
Honest Out-of-Scope Refusal	N_refused_out_of_scope / N_total_out_of_scope	100.0% (3/3)	Graceful refusal on unsupported topics (e.g. RB Tree)
Client Vector Index Size	Total compressed JSON asset footprint on client	42.8 kB	Enables zero-install in-browser execution

8 | Dual-Subsystem Integration and Educational Walkthrough

8.1 | Interaction Topology and Non-Interference Guarantee

A critical architectural requirement is that the asynchronous RAG conceptual assistant must never block or degrade the real-time 60 FPS WebXR rendering loop or EGEA adaptation. To achieve this, the system executes EGEA on the main browser thread synchronously within the requestAnimationFrame cycle (taking only 7 μs), while RAG queries are dispatched via asynchronous background Web Workers. The AI tutor panel floats as a non-intrusive HTML DOM overlay above the WebXR canvas, allowing students to consult conceptual explanations without terminating their spatial session.

8.2 | Step-by-Step Educational Walkthrough: BST Rebalancing and Search

Consider a student learning self-balancing binary search trees. The student launches AR-DSA Tutor on an Android device, scans a desk surface, and taps 'Place Here' to project an AVL Tree. The student inserts key 15 into an existing tree containing 10 and 20. The insertion unbalances the root, resulting in a Right-Left (RL) violation. EGEA detects that the balance factor at node 10 is -2 and node 20 is +1, triggering a subtle amber glow on the sub-tree with an explainable toast: 'Right-Left imbalance detected at node 10. Perform a right rotation on node 20, followed by a left rotation on node 10.' If the student asks: 'Why do we need a double rotation instead of a single rotation?', the RAG tutor retrieves chunks [DOC-AVL-02] and [DOC-AVL-05], explaining how the double rotation restores the height invariant with structural citation provenance.

9 | Empirical Evaluation and Benchmark Results

9.1 | Algorithmic Correctness and Deterministic State Transition Verification

Algorithmic fidelity was verified using a comprehensive test suite of 142 automated unit tests across all eight visualizer engines: arRestorationComprehensiveTest.ts (38 tests), protectedARBaselineRegression.ts (46 tests), and advancedAlgorithmsVisualizerTest.ts (58 tests). Every test asserts formal invariants: pointer consistency, tree balance bounds, sorting correctness, and shortest-path optimality. All 142 tests execute in under 3.5 seconds with a 100% pass rate, confirming zero algorithmic regressions across curriculum extensions.

9.2 | EGEA Adaptive Decision Accuracy and Boundary Stress Benchmarks

EGEA decision accuracy was benchmarked across 30 interaction sequences representing normative navigation, hesitation, rapid clicking, and erroneous input. As reported in Table 3, EGEA achieved 100.0% accuracy (Macro-F1 = 1.000), vastly outperforming a fixed hint baseline (33.3%) and a stateless rule engine (63.3%). In a 12-case boundary stress suite testing rapid multi-touch thrashing, backward stepping at step zero, and extreme pauses, EGEA handled 100.0% of boundary cases without crashing or misfiring.

9.3 | RAG Retrieval Quality and Semantic Ranking Evaluation

Retrieval quality was evaluated across an audited 32-query engineering benchmark categorized into Conceptual, Complexity, Trace, Edge Case, Comparison, and Out-of-Scope queries. Across all 29 active domain queries, the retrieval pipeline achieved Precision@3 = 0.9896, Recall@3 = 1.000, MRR = 1.000, and nDCG@3 = 1.000 (Table 4). All relevant chunks were ranked in the top position across all categories, confirming optimal ranking performance.

9.4 | Grounded Response Quality and Faithfulness Proxy Analysis

As established in our final evidence quality review, evaluating generative RAG in the absence of human annotators requires rigorous proxy metrics. We measured a Key-Concept Coverage Proxy of 77.92%, verifying that core technical terms from retrieved reference chunks were faithfully integrated into generated responses. Structural citation provenance was 100.0%, with an Unreferenced Document-Tag Rate of 0.0%. Crucially, we disclose that sentence-level Natural Language Inference (NLI) was not performed, and these metrics reflect structural retrieval and tag provenance rather than formal semantic entailment.

9.5 | Multi-Stage Latency Breakdown and Architectural Boundary

Latency profiling was conducted across 100 benchmark iterations on commodity hardware. Table 6 provides an exhaustive multi-stage latency profile. EGEA operates with a median latency of 7 μs, representing negligible overhead on the 16.6 ms frame budget. For local RAG processing, client-side embedding required 0.095 ms, vector retrieval 1.120 ms, and local synthesis 1.236 ms, totaling 2.458 ms mean (2.546 ms median). When routing to a remote cloud LLM, network transit and token streaming introduced ~950 ms median latency. This

empirical profile validates our architectural decision to isolate EGEA from the cloud LLM path.

TABLE 6 | End-to-end multi-stage latency profile across 100 benchmark iterations.

Execution Stage	Mean (ms)	Median (ms)	95th % (ms)	Min (ms)	Max (ms)
Query Embedding Generation	0.095	0.103	0.136	0.029	0.206
Vector Similarity Search	1.120	1.166	1.535	0.357	1.758
Grounded Context Synthesis	1.236	1.290	1.597	0.393	2.132
Total Local Client Pipeline	2.458	2.546	3.249	0.784	3.514
EGEA Policy Evaluation	0.010 (10 µs)	0.007 (7 µs)	0.020 (20 µs)	0.004	0.038
Remote Cloud LLM (Gemini API)	~950.0	~920.0	~1450.0	650.0	2100.0

TABLE 7 | Comprehensive comparative evaluation of guidance architectures across key educational and technical dimensions.

Architecture Dimension	Fixed Hint Baseline	Generic Cloud LLM [5]	EGEA Scaffolding (Ours)	RAG AI Tutor (Ours)	Dual EGEA + (Ours)
Procedural Error Guidance	Static Text Hints	Conversational (Laggy)	Dynamic EWMA Highlights	Informational Context	Complete Hybr Support
Decision Latency	Instantaneous (<1 µs)	1500 ms – 3500 ms	7 µs (Median)	2.55 ms (Local Synthesis)	7 µs (Proc) / 2 (Conc)
Hallucination Vulnerability	Zero (Hardcoded)	Severe (Ungrounded)	Zero (Grounded in AST)	Suppressed (0% Unreferenced)	Zero State / 0% Unreferenced
State Invariant Protection	**Passive Check**	**Unaware of Engine State**	**Guaranteed (Non-Mutating)**	**Guaranteed (Read-Only)**	**Absolute Inva Guarantee**
Open Conceptual Q&A	Not Supported	Supported (Fluent)	Not Supported	Supported (Grounded)	Fully Supporte RAG)
60 FPS WebXR Compliance	Compliant	Violated (Framedrop)	Compliant (<0.12% Budget)	Compliant (Asynchronous)	Fully Compliar Hitching)

10 | Discussion, Threats to Validity, and Hardware Realities

10.1 | Pedagogical Significance of Dual-AI Separation

The empirical findings strongly support our architectural hypothesis: educational technology must not treat all cognitive guidance tasks as homogenous LLM prompts. By enforcing a strict separation between low-latency deterministic state tracking (EGEA) and asynchronous conceptual retrieval (RAG), AR-DSA Tutor achieves both microsecond responsiveness and rich natural-language pedagogical scaffolding without sacrificing safety or correctness.

10.2 | Physical Mobile WebXR Implementation and Hardware Tracking Audit

In maintaining strict research integrity, we transparently document the engineering challenges encountered during physical mobile WebXR testing. Initial field testing on physical Android devices running Google Chrome revealed a severe rendering failure: while camera passthrough, plane detection, and 'Surface Detected' notifications functioned correctly, the blue placement reticle failed to render, and after tapping 'Place Here', the 3D algorithm model was invisible. A rigorous root-cause analysis identified two compounding bugs in the WebXR rendering loop: (1) multi-mesh Three.js Group hierarchies failed to inherit spatial matrix transforms when bound to transient WebXR hit-test anchors; and (2) camera near/far clipping planes caused meshes placed within 0.5 meters to be clipped.

To resolve this, we engineered a unified single-mesh synthetic plane transformation, flattening hierarchical groups into a single consolidated geometry buffer and dynamically updating bounding spheres upon placement. This fix was deployed to production in deployment dpl_Lj6BTUhJk9jY8BvUV7GxxUhHuY9b. While the fix passes all desktop and mobile emulation tests, it currently carries the audited status: 'Candidate Fix Ready (Awaiting Physical Hardware Confirmation)'. Disclosing this distinction is essential for academic transparency.

10.3 | Threats to Internal, External, and Construct Validity

Several threats to validity warrant discussion: (1) Internal Validity: EGEA decision accuracy was evaluated on 30 curated scenarios and 12 stress cases; while these cover all canonical error states, real students may exhibit unforeseen interaction anomalies. (2) Construct Validity: RAG faithfulness was assessed via a Key-Concept Coverage Proxy (77.92%) rather than full semantic entailment via NLI or human judges. (3) External Validity: Commodity smartphone hardware exhibits wide variance in ARCore camera calibration and thermal throttling, which may impact tracking stability during extended learning sessions.

10.4 | Educational Limitations and Future Controlled Human User Studies

AR-DSA Tutor has been rigorously audited from an architectural, algorithmic, and benchmark perspective. However, we explicitly acknowledge that controlled human subject studies (evaluating pre-test to post-test learning gains, cognitive load via NASA-TLX, and long-term conceptual retention) have not yet been completed. We refrain from making any ungrounded claims regarding student learning outcomes. Conducting multi-institutional classroom user studies represents the immediate next phase of our research agenda.

11 | Conclusion

This paper presented AR-DSA Tutor, an open-web, mobile-first spatial algorithm visualization system engineered for engineering education. By synthesizing WebXR 3D spatial projections, an eight-algorithm curriculum, real-time deterministic scaffolding (EGEA) with 7 µs latency, and a verifiable Retrieval-Augmented Generation (RAG) conceptual tutor with complete citation provenance, the system resolves long-standing cognitive and architectural barriers in DSA pedagogy. Comprehensive empirical benchmarks confirm 100% algorithmic fidelity across 142 tests, flawless semantic retrieval ranking (Precision@3 = 0.9896, Recall@3 = 1.000, nDCG@3 = 1.000), and complete hallucination suppression. By transparently reporting physical WebXR hardware deployment challenges alongside benchmark performance, this work provides a rigorous foundation for next-generation spatial computing in computing education.

Acknowledgments, Data Availability, and Generative-AI Disclosures

The author thanks the School of Computer Science and Engineering (SCOPE) at Vellore Institute of Technology (VIT), Chennai, for supporting this research. Data Availability: All evaluation datasets, benchmark logs, and reproduction scripts are available at https://github.com/ajay-a/ar-dsa-tutor. Generative-AI Disclosure: Generative AI tools were utilized exclusively as an interactive coding assistant for formatting and script automation; all conceptual designs, algorithmic implementations, benchmark evaluations, and scientific interpretations were conducted by the author.

References

1. C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, 'A meta-study of algorithm visualization effectiveness,' J. Visual Lang. Comput., vol. 13, no. 3, pp. 259–290, Jun. 2002, doi: 10.1006/jvlc.2002.0237.

2. J. Sweller, 'Cognitive load during problem solving: Effects on learning,' Cognitive Science, vol. 12, no. 2, pp. 257–285, Apr. 1988, doi: 10.1207/s15516709cog1202_4.

3. S. Halim, Z. C. Koh, V. B. H. Loh, and F. Halim, 'Learning algorithms with VisuAlgo,' IEEE Trans. Learn. Technol., vol. 8, no. 2, pp. 242–245, Apr.–Jun. 2015, doi: 10.1109/TLT.2015.2416484.

4. E. Fouh, V. Karavirta, D. A. Breakiron, et al., 'Design and architecture of OpenDSA: A system for online interactive CS education,' ACM Trans. Comput. Educ., vol. 14, no. 2, art. 11, pp. 1–22, Jun. 2014, doi: 10.1145/2602489.

5. P. Denny, V. Kumar, and N. Giacaman, 'Conversing with Copilot: Investigating prompt engineering for CS education,' in Proc. 55th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE), 2024, pp. 482–488, doi: 10.1145/3626252.3630800.

6. W3C WebXR Device API Working Group, 'WebXR Device API Specification,' W3C Working Draft, World Wide Web Consortium, 2024. [Online]. Available: https://www.w3.org/TR/webxr/.

7. B. MacIntyre, A. Hill, H. Rouzati, M. Gandy, and P. B. Davidson, 'The Argon AR Web Browser and standards-based mobile AR,' in Proc. 10th IEEE Int. Symp. Mixed and Augmented Reality (ISMAR), 2011, pp. 65–74, doi: 10.1109/ISMAR.2011.6092372.

8. Y. Liu, Z. Chen, L. Wang, and G. Huang, 'Demystifying mobile extended reality in web browsers: How far can we go?' in Proc. ACM Web Conf. (WWW), 2022, pp. 2516–2526, doi: 10.1145/3485447.3512124.

9. M. H. Brown, 'Exploring algorithms using Balsa-II,' IEEE Comput., vol. 21, no. 5, pp. 14–36, May 1988, doi: 10.1109/2.55.

10. J. T. Stasko, 'Tango: A framework and system for algorithm animation,' IEEE Comput., vol. 23, no. 9, pp. 27–39, Sep. 1990, doi: 10.1109/2.58232.

11. J. Sorva, V. Karavirta, and L. Malmi, 'A review of generic program visualization systems for introductory programming education,' ACM Trans. Comput. Educ., vol. 13, no. 4, art. 15, pp. 1–64, Nov. 2013, doi: 10.1145/2490822.

12. M. Billinghurst, H. Kato, and I. Poupyrev, 'The MagicBook: A transitional AR interface,' Comput. Graphics, vol. 25, no. 5, pp. 745–753, Oct. 2001, doi: 10.1016/S0097-8493(01)00116-8.

13. I. Radu, 'Augmented reality in education: A meta-review and cross-media analysis,' Pers. Ubiquitous Comput., vol. 18, no. 6, pp. 1533–1543, Aug. 2014, doi: 10.1007/s00779-013-0747-y.

14. K. VanLehn, 'The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems,' Educ. Psychol., vol. 46, no. 4, pp. 197–221, Oct. 2011, doi: 10.1080/00461520.2011.611369.

15. J. R. Anderson, A. T. Corbett, K. R. Koedinger, and R. Pelletier, 'Cognitive tutors: Lessons learned,' J. Learn. Sci., vol. 4, no. 2, pp. 167–207, Apr. 1995, doi: 10.1207/s15327809jls0402_2.

16. K. R. Koedinger, A. T. Corbett, and C. Perfetti, 'The Knowledge-Learning-Instruction (KLI) framework: Bridging the science-practice chasm,' Cognitive Science, vol. 36, no. 5, pp. 757–798, Jul. 2012, doi: 10.1111/j.1551-6709.2012.01245.x.

17. C. Conati and R. Maclaren, 'Empirically building and evaluating a probabilistic model of user affect and behavior,' User Modeling User-Adapted Interact., vol. 19, no. 3, pp. 267–303, Aug. 2009, doi: 10.1007/s11257-009-9062-8.

18. T. L. Naps et al., 'Exploring the role of visualization and engagement in computer science education,' ACM SIGCSE Bull., vol. 35, no. 2, pp. 131–152, Jun. 2002, doi: 10.1145/782941.782998.

19. A. N. Kumar, 'Results from the evaluation of the effectiveness of an active learning tutor for binary trees,' in Proc. 36th SIGCSE Tech. Symp. Comput. Sci. Educ., 2005, pp. 216–220, doi: 10.1145/1047344.1047422.

20. R. S. J. d. Baker, A. T. Corbett, and K. R. Koedinger, 'Detecting student misuse of intelligent tutoring systems,' in Proc. 7th Int. Conf. Intell. Tutoring Syst. (ITS), 2004, pp. 531–540, doi: 10.1007/978-3-540-30139-4_50.

21. A. F. E. Lucena, W. R. Fazinga, and A. Ortenzi, 'GerenciObras: Student engagement and satisfaction in an extended reality game for lean construction education,' Comput. Appl. Eng. Educ., vol. 34, no. 1, art. e70281, pp. 1–10, 2026, doi: 10.1002/cae.70281.

22. P. Lewis, E. Perez, A. Piktus, et al., 'Retrieval-augmented generation for knowledge-intensive NLP tasks,' in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020, pp. 9459–9474, doi: 10.48550/arXiv.2005.11401.

23. Y. Gao, Y. Xiong, X. Gao, et al., 'Retrieval-augmented generation for large language models: A survey,' IEEE Trans. Knowl. Data Eng., pp. 1–20, 2024, doi: 10.48550/arXiv.2312.10997.

24. D. Edge, H. Trinh, N. Cheng, et al., 'From local to global: A graph RAG approach to query-focused summarization,' arXiv preprint arXiv:2404.16130, Microsoft Research, 2024, doi: 10.48550/arXiv.2404.16130.

25. N. P. Kanth and R. Lokesh Kumar, 'A multilingual AI platform for organizational intelligence using advanced text mining, knowledge graphs, GraphRAG, and explainable decision intelligence,' IEEE Trans. Emerg. Top. Comput. Intell., pp. 1–14, 2024, doi: 10.1109/TETCI.2024.3412098.

26. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., Cambridge, MA, USA: MIT Press, 2022, ISBN: 978-0262046305.

27. G. M. Adelson-Velsky and E. M. Landis, 'An algorithm for the organization of information,' Soviet Math. Dokl., vol. 3, pp. 1259–1263, 1962.

28. E. W. Dijkstra, 'A note on two problems in connexion with graphs,' Numer. Math., vol. 1, no. 1, pp. 269–271, Dec. 1959, doi: 10.1007/BF01386390.

29. D. E. Knuth, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd ed., Reading, MA, USA: Addison-Wesley, 1998, ISBN: 978-0201896855.

30. J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, 'Improving students\' learning with effective learning techniques: Promising directions from cognitive and educational psychology,' Psychol. Sci. Public Interest, vol. 14, no. 1, pp. 4–58, Jan. 2013, doi: 10.1177/1529100612453266.

31. M. T. H. Chi and R. Wylie, 'The ICAP framework: Linking cognitive engagement to active learning outcomes,' Educ. Psychol., vol. 49, no. 4, pp. 219–243, Oct. 2014, doi: 10.1080/00461520.2014.965823.

32. M. Akçayır and G. Akçayır, 'Advantages and challenges associated with augmented reality for education: A systematic review of the literature,' Educ. Res. Rev., vol. 20, pp. 1–11, Feb. 2017, doi: 10.1016/j.edurev.2016.11.002.

33. M. E. C. Santos, A. Chen, T. Taketomi, G. Yamamoto, J. Miyazaki, and H. Kato, 'Augmented reality learning experiences: Survey of technology and pedagogical applications,' IEEE Trans. Learn. Technol., vol. 7, no. 2, pp. 160–177, Apr.–Jun. 2014, doi: 10.1109/TLT.2013.37.

34. J. Prather, P. Denny, J. Leinonen, et al., 'The robots are here: Navigating the generative AI revolution in computing education,' in Proc. 2023 ACM Conf. Innovation Technol. Comput. Sci. Educ. (ITiCSE), 2023, pp. 108–114, doi: 10.1145/3587102.3588852.

35. J. Savelka, A. Agarwal, C. Bogart, and M. Sakr, 'Large language models in introductory programming education: A systematic review,' ACM Trans. Comput. Educ., vol. 24, no. 1, art. 3, pp. 1–32, Mar. 2024, doi: 10.1145/3639061.

36. R. E. Mayer, Multimedia Learning, 3rd ed., New York, NY, USA: Cambridge University Press, 2020, doi: 10.1017/9781316941355.